



A Dynamic Approach for Template and Content Extraction in Websites

Nicolae Cristian-Catalin^(✉) and Mihaita Dragan

Faculty of Mathematics and Computer Science, University of Bucharest, Bucharest, Romania
nicolaecristiancatalin@gmail.com, mihaita.dragan@fmi.unibuc.ro

Abstract. Web scraping is a technique used to extract data from websites and it is the pillar of information retrieval in a world wide web that is ever growing. There are two main ways of extracting data from a website: static and dynamic scraping. Static scraping requires input beyond the target website because the user needs to inspect the HTML content of the target and find certain patterns in the templates that are then used to extract data. Static scraping is also very vulnerable to changes in the template of the web page. Dynamic scraping is a very broad topic and it has been tackled from many different angles: tree-based, natural language processing (NLP), computer vision or machine learning techniques. For most websites, the problem can be broken in two big steps: finding the template for the pages we want to extract data from and then removing irrelevant text such as ads, text from controls or JavaScript code.

This paper proposes a solution for dynamic scraping that uses AngleSharp for HTML retrieval and involves a slightly modified approach of the graph technique mentioned in for template finding. Once we find a number of pages then several heuristics can be applied for content extraction and noise filtering. Such heuristics can include: text and hyperlink density, but also removing common content between multiple pages (usually text from controls, static JavaScript) and then of final layer of NLP techniques (breaking the content into sentences, tokenization and part-of-speech tagging).

Keywords: Web scraping · Web graph topology · NLP · DOM content filtering

1 Introduction

The development of the World Wide Web has followed an exponential growth and the amount of information that is available is staggering. Humans can no longer parse all this information in their limited time and this problem gave birth to web scraping. The easier approach, from an algorithmic point of view, is static scraping. This kind of scraping requires a good amount of work in inspecting the targeted web pages and finding the template that enables us to retrieve information. One example would be noticing that relevant information is kept in <DIV> tags that have a specific class-name. Often times, with modern frameworks, these templates will have a much higher degree of complexity that requires more than just specific HTML patterns, but also specific code that is written for a certain web page, thus increasing the demands of static scraping.

Dynamic web scraping can be approached from many angles and several branches of computer science can be used in this regard. The first important step is obtaining the (Document Object Model) DOM. Many programming languages offer libraries that can parse and instantiate the DOM. In all browsers, when an HTML document is loaded into a web browser, the DOM is created. Then, every node is usable and ready to be filtered for the purpose of extracting information [5]. Early studies focused on the DOM-tree representation of Web pages and learn a template that wrap data records in HTML tags, such as [6, 7]. The most popular method is to use the structured, tree-like nature of the DOM [3] and to use known algorithms to extract patterns and data. Regular expressions are also a common tool for the task since they can define a clear set of rules for content extraction. Computer vision can also be used to allow information retrieval in a similar way to humans. This approach involves rendering the DOM and then using several heuristics like pixel distance to the top of the page or font size to identify the title, splitting the page into different zones and narrowing the search to the central area and many others. Machine learning is another excellent approach that yields great results since manually labeled web pages can be used as a training set in a supervised learning task [3]. The most successful solutions often use a combination of the above.

The task of dynamic scraping can be broken into two big steps. The first step is finding several relevant pages that fit a certain template while starting from a main page. The second step is the task of extracting information from a HTML document with a minimum of noise (client-side JavaScript code, text from the controls in page such as “Follow us”). The first step is usually based on the oriented-graph nature of websites. The main page of a site can be considered the starting node and then the links to other pages can be considered edges. If a certain site contains a link that returns to our starting page, we have a bidirectional edge. Almost every modern website has a top header menu that is ever present on every other page. Pages that are linked by the main page and then link back to it will form a connected graph. Find the maximal connected graph (or a connected graph of a specified dimension) will give links to similar web pages from which we can start extracting a template. This is the approach proposed in [1] which does seem to give good results. For content extraction, a node level solution such as the one mentioned in [2] seems to offer a reduced amount of complexity with no sacrifice of correctness.

In Sect. 2, we present a few problems we’ve encountered with the algorithm in [1], used to find a complete subdigraph in a website topology. In Sect. 3 we discuss the methods that are used to extract the template from the complete subdigraph. In Sect. 4 we introduce the node-based filtering system that uses several heuristics, some simply based on node characteristic while others based on NLP processing. In Sect. 5 we summarize our approach and results and suggest further research such as NLP techniques that aggregate data in powerful ways.

2 Retrieving Relevant Web Pages

A good way of finding similar web pages is presented in [1] with a main page that is given as input. It relies on the strong heuristic that web pages are linked with the main page of a site through a menu. Together, these form a connected graph (because of the

implied bidirectional connections). The algorithm for finding a complete subdigraph in a website topology is explained in section 2 of [1]. The algorithms yields good results, but it does have some limitations:

- On starting pages that have a lot of outgoing links (large websites like BBC can have over 250) there is a very high complexity, admitted by the authors themselves and proposed as further work.
- Somewhat linked to the first issue, large websites use more than one template for their pages. For example, the BBC news site uses a template for articles, but it also uses a template for categories (e.g. BBC Sport, BBC Travel, BBC Future). There is a high chance of encountering a subdigraph containing pages from another template than the one we expect (the one we can extract data from). A simple, yet effective solution to this issue is sorting the outgoing links by the length of their URLs in a descending manner. This is based on the fact that article links are always longer than most other links on a website, since they almost always contain the title of the article in the URL.

3 Extracting the Template

Once a subdigraph is found, the authors in [1] propose a tree based equal top-down approach of extracting the template. This approach is very precise, but can be somewhat rigid for most websites since templates don't always match node by node. The alternative approach that we have used is to a frequency array for every type of HTML element in the respective DOM. We can pad accordingly with the missing HTML elements between them so all frequency arrays have the same size and elements, just different values. We can then calculate the standard deviation for each HTML element (with a dataset of four, in our example) and then average all these value into one single value that tell us how strongly related the web pages in the subdigraph really are (Fig. 1).

```

01. // Compute the frequency dictionary for every DOM.
02. var pagesFrequencyDictionaryList = webDocuments
03.   .Select(dom => dom.All.GroupBy(element => element.GetType().ToString())
04.     .ToDictionary(x => x.Key, x => x.Count()).ToList());
05. // <...> <Pad every frequency array with
06. // missing tags from other arrays> <...>
07. var tagsArray = allHTMLTags.ToArray();
08. for (int iTagIdx = 0; iTagIdx < allHTMLTags.Count; iTagIdx++)
09. {
10.     // Get all values for this particular HTML tag.
11.     List<int> tagValues = new List<int>();
12.     foreach (var pageDictionary in pagesFrequencyDictionaryList)
13.         tagValues.Add(pageDictionary[tagsArray[iTagIdx]]);
14.     // Compute median and standard deviation for the list of values.
15.     templateFrequency[tagsArray[iTagIdx]] = tagValues.GetMedian();
16.     templateStandardDeviation[tagsArray[iTagIdx]] =
17.         tagValues.GetStandardDeviation();
18. }
19. double averageStandardDeviation = templateStandardDeviation.Values.Sum() /
20.     templateStandardDeviation.Values.Count;

```

Fig. 1. First compute a list of dictionaries (frequency arrays), pad them with missing values, get the list of values for each individual tag, compute standard deviation for all tags and average this set of values for a good index of how suitable this graph is for extracting a template.

We can set a threshold for this value so we can reject web pages that do form a complete subdigraph, but their content is largely unrelated. Once a subdigraph is validated by this threshold value, like we used the standard deviation of every HTML element in the frequency array, we can compute the median count for each element (the median is more resilient to outliers than the average and we want to be forgiving to certain elements being present in one instance of the template and not another). This array of median values will be the final template. One disadvantage of this approach is the fact that is losing the connections between the nodes present in the DOM. The advantage is that the approach is forgiving to variations between instances of the template and it is fast to compute.

4 Filtering the Content of the Web Page

Even after parsing the DOM, there is still a lot of work to be done in order to have the main content from the page. The approach in [2] computes two values for every node: text density and hyperlink density. A first filter would only keep all HTML that are known to store information: divs, paragraphs, table cells and so on.

Then, one heuristic that is used is that a content node has higher text density and lower hyperlink density than the noisy node. The second heuristic used is the fact that nodes are grouped so nodes that are adjacent will be of a similar type. This approach does remove a good amount of noisy nodes, but in our experiments it is not powerful enough to certain cases:

- Nodes that contain JavaScript code can have a high text density and a low hyperlink density, so the first heuristic can have such false negatives.
- We can have nodes that sit at the border between a group of content nodes and a group of noisy nodes, which can pass through the second heuristic.
- Text from the controls and labels in the GUI tend to pass both heuristics (text such as “Follow us on Instagram”, “Subscribe on Youtube”).

One problem we’ve encountered is that fixed thresholds for text density and hyperlink density can be very prohibitive and will easily erase all relevant content. This is caused by the fact that not every website holds its content scattered among many HTML elements, so we will encounter a very large node with a low text density, but this node can actually be very relevant. A good method is to compute the thresholds at runtime, using the average value for text density and hyperlink density in the current document.

After this step, one method that can easily get rid of text from controls is taking the intersection of nodes with the same content between the pages in the subdigraph. This is, of course, based on the observation that the same buttons will be present on each page and they will figure as common content, but the main content will differ and we won’t be removing parts of it. Even after this step, JavaScript nodes that contain hashes and code that varies will remain. The final step is to use NLP techniques [4]. We can break all content we have so far into sentences, tokenize the sentences into words and then tag the words into parts of speech. For NLP, we are using the C# port of OpenNLP. After these three steps, we can use two heuristics:

- Every content sentence must contain at least one verb. The part of speech tagger in OpenNLP has a best effort approach to tagging so we will always have tags, even on nonsensical sentences generated by JavaScript nodes. Nodes that do not have at least one verb can be removed.
- JavaScript nodes usually contain long strings such as hashes. We can simply remove every node that contains a word that is longer than a certain threshold.

The NLP approach is quite powerful and has the advantage of computing tokenized, tagged sentences that can be further used in information retrieval. A stronger, but very costly approach would be trying every word against a dictionary of the respective language. While this would almost all noisy nodes, the computational cost of checking every word against a dictionary would be enormous. A small disadvantage of adding NLP to this problem is that articles we can extract information from are now reduced to being written in languages we are trained to process.

5 Testing the Results

A good method for testing the solution is Levenshtein distance, also known as edit-distance, the number of edits required to go from one string to another. We can regard these edits as errors or missing information.

This will be accomplished by comparing the output content of the algorithm with a hand-gathered, perfectly scraped output. To remove several irrelevant edits, we can ignore newlines and white spaces. A good metric could also be the Levenshtein distance divided by the length of the perfectly scraped article since it shows how much of the article is missing or how much noise is left.

	Perfectly scraped output size	Solution output size	Levenshtein distance	Distance over length factor
HackerNews article 1	5600	5792	924	0.165
HackerNews article 2	3272	3639	663	0.202
HackerNews article 3	3861	4283	608	0.157
HackerNews article 4	5011	5102	1090	0.217
The Guardian article 1	6107	6902	1586	0.259
The Guardian article 2	5410	5940	850	0.155
Blog article 1	5948	6192	1251	0.21
Blog article 2	4500	4689	1064	0.23

As it can be seen, the algorithm performs within reasonable parameters and while the node content filtering could be a bit more severe in order to reduce the noise even more, doing that would risk removing relevant content, which is not preferred.

6 Conclusion

The topic of web scraping is already very developed and current technologies are very powerful. The proposed solution combines a few branches of computer science and mathematics: graph theory, data science and NLP. The solution performs well as long as it gets the template for news article right, this being the problematic part of most scraping solutions since large websites have so many outgoing links it's very easy to get side-tracked. The template extraction method is a bit more flexible than tree-based approaches and is computed in a very fast manner using simple frequency arrays measured in terms of standard deviation and average. Node filtering is performing adequately. Severe filtering would lead to a lot of content being removed so false positives (as in, remaining JavaScript or other noise) is preferred to false negatives (content being tagged as noise), but only in reasonable terms.

Further work could go towards improving the algorithm on large websites where it still gets off-topic or creating a system that can deduce reasonable thresholds on the go (some websites can yield zero data since every node is removed in the node filtering process). Another direction for further work, since we're using NLP techniques and the data from the last step of node filtering, we can maybe break the content into a system of knowledge representation and create a chat bot that can process questions in natural language, query the knowledge base and answer in natural language.

References

1. Alarte, J., Insa, D., Silva, J., Tamarit, S.: Web template extraction based on hyperlink analysis. In: Escobar, S. (ed.): XIV Jornadas sobre Programación Y Lenguajes, PROLE 2014, Revised Selected Papers EPTCS, vol. 173, pp. 16–26 (2015)
2. Liu, Q., Shao, M., Wu, L., Zhao, G., Fan, G.: Main content extraction from web pages based on node characteristics. *J. Comput. Sci. Eng.* **11**(2), 39–48 (2017)
3. Ferrara, E., De Meob, P., Fiumarac, G., Baumgartner, R.F.: Web Data Extraction, Applications and Techniques: A Survey. [arXiv:1207.0246v4](https://arxiv.org/abs/1207.0246v4) [cs.IR], 10 June 2014
4. OpenNLP Documentation. <https://opennlp.apache.org/docs/>. Accessed 2 Nov 2019
5. Uzun, E., Doruk, A., Nusret Buluş, H., Özhan, E.: Evaluation of HAP, AngleSharp and HTML document in web content extraction. In: International Scientific Conference, Gabrovo, 18 November 2017 (2017)
6. Soderland, S.: Learning information extraction rules for semi-structured and free text. *Mach. Learn.* **34**(1–3), 233–272 (1999)
7. Muslea, I., Minton, S., Knoblock, C.: Hierarchical wrapper induction for semistructured information sources. *Auton. Agents Multi-Agent Syst.* 93–114 (2001). <https://doi.org/10.1023/A%3A1010022931168>